Informe de auditoría funcional y de seguridad con Selenium — Konny's Pizza

RESULTADOS DE LA AUDITORÍA

44 COMPROBACIONES	**38** APROBADAS	**6** BLOQUEADAS CORRECTAMENTE
0 FALLIDAS	**0** ERRORES DE AUTOMATIZACIÓN	**0** OMITIDAS / NO VERIFICADAS

1. Datos generales

Campo	Valor
Proyecto	Konny's Pizza — aplicación web de reservas
Objetivo	Verificar autenticación administrativa, no persistencia de sesión, cierre y reinicio por inactividad, API pública, navegación, validaciones de reserva y RLS anónima
Herramientas	Selenium WebDriver 4.52.0 (Node.js), Firefox headless, geckodriver (Selenium Manager), Node.js v26.11.1, `curl`, Git
URL evaluada	https://konys-pizza.vercel.app y https://konys-pizza.vercel.app/admin
Fecha y hora de ejecución	8 de noviembre de 2026, 16:35:33–16:41:56 y 16:55:15–16:55:34; 19:21:33–19:31:28 y 19:34:41 (America/Bogota, UTC–5)
Alcance de esta ejecución	Fase 1 (comprobaciones 1–7: login, panel, almacenamiento, inactividad, cierre, recuperación y evidencias) y fase 2 (API anónima, navegación pública, validaciones de reserva y reinicio del contador)
Identidad utilizada	Cuenta administrativa autorizada. Su correo y contraseña no se registran en este informe ni en las evidencias
Escrituras realizadas	Solo las inherentes a la autenticación, al heartbeat, al cierre de sesión y a las consultas RPC de diagnóstico, autorizadas por el propietario
Datos de negocio modificados	Ninguno. No se crearon, editaron ni borraron reservas, mesas, platos, horarios ni políticas RLS

> *Las credenciales se inyectaron únicamente como variables de entorno durante la ejecución. No se escribieron en scripts, archivos, capturas, logs, informes ni commits.*

---

2. Resumen ejecutivo

Se ejecutaron **44 comprobaciones**: 9 de la fase 1 y 35 de la fase 2. El resultado fue:

Estado	Cantidad
APROBADA	38
FALLIDA	0
BLOQUEADA CORRECTAMENTE	6
ERROR DE AUTOMATIZACIÓN	0
OMITIDA	0
NO VERIFICADA	0

Fase	Comprobaciones	APROBADA	BLOQUEADA CORRECTAMENTE	FALLIDA
Fase 1: autenticación e inactividad	9	9	0	0
Fase 2: API, navegación, validaciones y reinicio	35	29	6	0
Total	**44**	**38**	**6**	**0**

Conclusión del alcance ejecutado: el panel administrativo exige autenticación, no persiste tokens en `localStorage`, `sessionStorage` ni cookies, cierra la sesión tras cinco minutos sin actividad, la actividad válida reinicia el contador y no permite recuperar la sesión mediante un clic ni una recarga. La API anónima solo expone mesas, horarios, carta y ocupación sin datos personales; las tablas y RPC administrativas responden 401. La navegación pública y las validaciones de reserva funcionan y no invocan `crear_reserva` con datos inválidos.

Se detectó **un hallazgo de despliegue**: `sesion_admin_activa()` devuelve `401 permission denied` con un usuario autenticado, aunque el esquema local concede la función a `authenticated`. El heartbeat real (`registrar_actividad_admin`) y el cierre (`cerrar_sesion_admin`) sí responden 200. El hallazgo no afectó las comprobaciones de inactividad, pero **podría bloquear lecturas/escrituras administrativas sujetas a las políticas RLS** que invocan esa función; debe verificarse en un entorno aislado (sección 7).

Este informe **no cubre** todavía el recorrido completo de todas las vistas administrativas, las operaciones administrativas de escritura, las pruebas de RLS con un usuario autenticado no administrador ni las pruebas que requieren escribir datos. Esas pruebas quedan pendientes y se detallan en la sección 9.

---

3. Metodología

1. **Inspección de solo lectura** del repositorio local `Konny-s_Pizza`: páginas HTML, módulos JavaScript, pruebas Node/Selenium existentes, `vercel.json` y SQL de RLS.
2. **Verificación de cabeceras y exposición** con `curl`: CSP, HSTS, `X-Frame-Options`, `nosniff`, `Referrer-Policy`, `no-store` en `/admin`, `noindex` y respuesta 404 para archivos internos.
3. **Automatización con Selenium** sobre Firefox headless contra el despliegue de Vercel.
4. **Capturas de pantalla** en `docs/evidencias/`, tomadas solo cuando el formulario de acceso estaba vacío o cuando el panel estaba visible sin datos personales.
5. **Comprobación de almacenamiento** mediante `executeScript` sobre `localStorage`, `sessionStorage` y cookies, filtrando claves relacionadas con Supabase, tokens o autenticación.
6. **Espera real de 6 minutos y 15 segundos** sin eventos de usuario, sin comandos que generaran `mousemove`, `keydown`, `click`, `scroll` ni `touchstart`.
7. **Intento de recuperación** mediante clic en el formulario y recarga completa de la página.
8. **Fase 2 — API anónima:** consultas REST a `mesas`, `horarios`, `platos`, `reservas`, `administradores`, `sesiones_admin_actividad` y `obtener_ocupados`, más las RPC `es_admin` y `sesion_admin_activa`, usando solo la clave pública.
9. **Fase 2 — navegación pública:** recorrido de la página principal, menú móvil, pestañas de carta y modal de reserva, con capturas sin datos personales.
10. **Fase 2 — validaciones:** envío de datos inválidos al formulario de reserva, verificación de mensajes y contador de llamadas a `crear_reserva`; no se completó ninguna reserva.
11. **Fase 2 — reinicio del contador:** espera controlada, clic válido y medición del heartbeat y del cierre con instrumentación de `fetch` que solo registra ruta, estado y presencia de `Authorization`.

No se ejecutaron operaciones de escritura sobre datos de negocio: mesas, platos, horarios, reservas ni políticas RLS. Las únicas escrituras fueron las de autenticación, heartbeat y cierre de sesión.

---

4. Resultados detallados

C1.0 — Abrir `/admin` sin sesión

- **Procedimiento:** abrir `https://konys-pizza.vercel.app/admin` en un navegador nuevo, sin cookies ni almacenamiento previo.
- **Resultado esperado:** formulario de acceso visible y panel administrativo oculto.
- **Resultado obtenido:** `#acceso` visible; `#panel` con `visible=false`.
- **Estado: APROBADA**.
- **Evidencia:** `docs/evidencias/01-acceso-sin-sesion.png`.

C1.1 — Iniciar sesión con la cuenta autorizada

- **Procedimiento:** completar correo y contraseña en el formulario y pulsar “Ingresar”.
- **Resultado esperado:** panel administrativo visible tras la autenticación.
- **Resultado obtenido:** el panel se mostró correctamente. No se mostró ningún aviso de error de autenticación.
- **Estado: APROBADA**.
- **Evidencia:** `docs/evidencias/02-panel-autenticado.png` (panel sin correo visible).

C2 — Comprobar que el panel administrativo se muestra correctamente

- **Procedimiento:** verificar la visibilidad del panel y de la vista “Mesas”, y comprobar que la tabla de mesas cargó.
- **Resultado esperado:** panel y vista “Mesas” visibles, con filas de datos o estado vacío.
- **Resultado obtenido:** `#vista-mesas` visible; 6 filas en `#tabla-mesas`.
- **Estado: APROBADA**.
- **Evidencia:** `docs/evidencias/02-panel-autenticado.png`.

C3 — Verificar que credenciales o tokens no se guardan en `localStorage` ni `sessionStorage`

- **Procedimiento:** tras el login, leer las claves de `localStorage`, `sessionStorage` y las cookies; filtrar nombres relacionados con Supabase, `access_token`, `refresh_token` o autenticación.
- **Resultado esperado:** ninguna clave de autenticación ni cookie de sesión.
- **Resultado obtenido:** `localStorage=[]`, `sessionStorage=[]`, `clavesTipoAuth=[]`, `cookies=[]`.
- **Estado: APROBADA**.
- **Evidencia:** `docs/evidencias/03-almacenamiento.json`.

C4 — Dejar la sesión sin actividad durante al menos 6 minutos

- **Procedimiento:** tras el login y las comprobaciones de almacenamiento, mantener la sesión abierta sin eventos de usuario durante **6 minutos y 15 segundos**. No se ejecutaron clics, teclas, desplazamientos ni comandos que dispararan eventos del DOM.
- **Resultado esperado:** transcurrir el plazo sin actividad.
- **Resultado obtenido:** espera completada: 6 minutos 15 segundos sin interacción.
- **Estado: APROBADA**.
- **Evidencia:** registro de ejecución y marcas de tiempo de las capturas 05–07.

C5 — Comprobar el cierre automático y la indisponibilidad del panel

- **Procedimiento:** al terminar la espera, comprobar si el formulario de acceso volvió a mostrarse, si el panel quedó oculto y qué mensaje apareció.
- **Resultado esperado:** formulario de acceso visible, panel oculto y aviso de cierre por inactividad.
- **Resultado obtenido:** `#acceso` visible; `#panel` oculto; aviso: “Tu sesión administrativa se cerró por cinco minutos de inactividad. Vuelve a ingresar.”

- **Estado: APROBADA**.
- **Evidencia:** `docs/evidencias/05-sesion-expirada.png`.

C6.1 — Intentar recuperar la sesión mediante una interacción

- **Procedimiento:** hacer clic en el campo de correo del formulario de acceso después del cierre y esperar 2 segundos.
- **Resultado esperado:** el panel sigue oculto y la sesión no se recupera.
- **Resultado obtenido:** `#panel` continúa oculto (`visible=false`).
- **Estado: APROBADA**.
- **Evidencia:** `docs/evidencias/06-tras-interaccion.png`.

C6.2 — Intentar recuperar la sesión mediante una recarga

- **Procedimiento:** recargar la página, esperar el formulario de acceso, comprobar el panel, el almacenamiento y ejecutar `haySesionAdmin()` desde el propio módulo de la aplicación.
- **Resultado esperado:** acceso visible, panel oculto, sin tokens y `haySesionAdmin()=false`.
- **Resultado obtenido:** acceso visible; panel oculto; `clavesTipoAuth=[]`; `cookies=[]`; `haySesionAdmin=false`.
- **Estado: APROBADA**.
- **Evidencia:** `docs/evidencias/07-tras-recarga.png`.

C7 — Registrar capturas y resultados sin datos sensibles

- **Procedimiento:** revisar las capturas y el JSON de resultados; comprobar que no aparecen correo, contraseña ni tokens.
- **Resultado esperado:** evidencias sin datos sensibles.
- **Resultado obtenido:** capturas 01, 02, 05–17 y JSON 03, 08–14; el formulario de acceso aparece vacío en las capturas; no se encontró el correo ni la contraseña en `docs/evidencias/`.
- **Estado: APROBADA**.
- **Evidencia:** `docs/evidencias/`.

Fase 2 — API anónima, navegación, validaciones y reinicio del contador

F2.1 — API pública con identidad anónima

Se consultó la API REST de Supabase sin sesión, usando únicamente la clave pública del cliente. No se usaron credenciales administrativas.

ID	Consulta	Resultado obtenido	Estado
API-01	Leer `mesas`	200; 2 filas; columnas públicas	APROBADA
API-02	Leer `horarios`	200; 2 filas	APROBADA
API-03	Leer `platos`	200; 2 filas	APROBADA
API-04	Leer `reservas`	401; `permission denied`	BLOQUEADA CORRECTAMENTE
API-05	Leer campos personales de reservas	401	BLOQUEADA CORRECTAMENTE
API-06	Leer `administradores`	401	BLOQUEADA CORRECTAMENTE
API-07	Leer `sesiones_admin_actividad`	401	BLOQUEADA CORRECTAMENTE
API-08	Leer `obtener_ocupados`	200; columnas `mesa_id`, `hora_inicio`, `hora_fin`; 1 fila	APROBADA
API-09	Invocar `es_admin()` como anónimo	401	BLOQUEADA CORRECTAMENTE

ID	Consulta	Resultado obtenido	Estado
API-10	Invocar `sesion_admin_activa()` como anónimo	401	BLOQUEADA CORRECTAMENTE

- **Evidencia:** `docs/evidencias/08-api-anonima.json`.
- **Conclusión:** la API anónima no expone reservas, datos personales, administradores ni sesiones. La ocupación pública se limita a mesa y franja horaria.

F2.2 — Navegación pública

ID	Acción	Resultado obtenido	Estado	Evidencia
NAV-01	Cargar la página principal	Título correcto; 6 mesas, 17 platos y 3 pestañas de carta	APROBADA	`08-home-desktop.png`
NAV-02	Verificar horario de hoy y pie de página	Textos de horario y contacto visibles	APROBADA	`08-home-desktop.png`
NAV-03	Abrir el menú móvil	Menú visible, `aria-expanded=true` y clase abierta	APROBADA	`09-menu-movil.png`
NAV-04	Cerrar el menú móvil	Menú cerrado y `aria-expanded=false`	APROBADA	
NAV-05	Recorrer las pestañas de la carta	Cada pestaña muestra sus platos y queda seleccionada	APROBADA	`10-carta.png`
NAV-06	Abrir el modal de reserva	Título, calendario y franjas visibles	APROBADA	
NAV-07	Seleccionar una franja libre	Resumen con la hora elegida	APROBADA	`11-modal-reserva.png`
NAV-08	Cambiar de mes en el calendario	El mes mostrado cambia	APROBADA	
NAV-09	Cerrar el modal	Modal oculto	APROBADA	

- **Evidencia estructurada:** `docs/evidencias/09-navegacion-publica.json`.
- **Incidencia de automatización corregida:** la primera versión usaba referencias obsoletas a las pestañas de carta y produjo `StaleElementReferenceError`. Se corrigió para releer cada pestaña antes de interactuar. No fue un fallo de la aplicación.

F2.3 — Validaciones del formulario de reserva

En los casos de formulario inválido (VAL-01..09) se comprobó que **no se invocó** `crear_reserva` (`rpc=0`). En VAL-10 y VAL-11 el aviso aparece al elegir la franja, antes de enviar el formulario.

ID	Caso	Resultado obtenido	Estado	Evidencia
VAL-01	Enviar el formulario vacío	Errores de nombre y personas; `rpc=0`	APROBADA	`12-validacion-campos.png`
VAL-02	Nombre de una sola letra	Error de mínimo 2 caracteres; `rpc=0`	APROBADA	`13-validacion-nombre.png`
VAL-03	Nombre con etiquetas HTML	Error de caracteres no permitidos; `rpc=0`	APROBADA	
VAL-04	Teléfono con letras	Error de teléfono; `rpc=0`	APROBADA	`14-validacion-telefono.png`
VAL-05	Teléfono demasiado corto	Error de teléfono; `rpc=0`	APROBADA	
VAL-06	Teléfono válido con otro campo inválido	Error de personas; `rpc=0`	APROBADA	
VAL-07	Personas por encima de la capacidad	Error de máximo según la mesa; `rpc=0`	APROBADA	`15-validacion-capacidad.png`

ID	Caso	Resultado obtenido	Estado	Evidencia
VAL-08	Personas con decimales	Error de número entero; `rpc=0`	APROBADA	
VAL-09	Personas en cero	Error de número; `rpc=0`	APROBADA	
VAL-10	Franja ya ocupada	Aviso de franja ocupada	APROBADA	
VAL-11	Franja pasada	Aviso de hora pasada	APROBADA	

- **Evidencia estructurada:** `docs/evidencias/10-validaciones-reserva.json` y `docs/evidencias/12-resumen-fase2-publica.json`.
- **Conclusión:** las validaciones del cliente bloquean los datos inválidos antes de llamar a la RPC. No se creó ninguna reserva.

F2.4 — Reinicio del contador por actividad válida

Se instrumentó `window.fetch` para registrar únicamente la ruta, el estado HTTP y si la petición llevaba cabecera `Authorization`; no se guardaron cabeceras, tokens ni cuerpos.

ID	Acción	Resultado obtenido	Estado	Evidencia
ACT-01	Iniciar sesión	Panel visible; acceso oculto	APROBADA	
ACT-02	Esperar 4 min 10 s sin actividad	Panel visible; sin heartbeats durante la espera	APROBADA	
ACT-03	Hacer un clic válido a los 4 min 10 s	Panel visible; `registrar_actividad_admin` responde 200	APROBADA	
ACT-04	Esperar 6 min desde el login (2 min desde el clic)	Panel visible; heartbeat 200; no se cerró a los 5 min del login	APROBADA	`16-admin-tras-reinicio.png`
ACT-05	Esperar 5 min 15 s desde el clic	Panel oculto; acceso visible; aviso de cierre; c `errar_sesion_admin` responde 200	APROBADA	`17-admin-cierre-final.png`

- **Evidencia estructurada:** `docs/evidencias/11-reinicio-inactividad.json`.
- **Conclusión:** la actividad válida renueva el heartbeat y reinicia el plazo; el cierre se produce cinco minutos después de la última actividad, no cinco minutos después del login.
- **Incidencia de automatización corregida:** la primera versión de ACT-04 usó `haySesionAdmin()` como criterio y falló por el 401 de `sesion_admin_activa()`. El diagnóstico mostró que el heartbeat real seguía respondiendo 200; la versión final valida `registrar_actividad_admin` y el estado del panel.

F2.5 — Diagnóstico de sesión y RPC administrativas

Prueba	Resultado obtenido	Interpretación	Evidencia
Tras el login, `getSession()`	`session=false`	El helper `haySesionAdmin()` no es fiable en este despliegue	`13-diagnostico-sesion-admin.json`
Tras el login, `getUser()`	`Auth session missing!`	Igual que el anterior	`13-diagnostico-sesion-admin.json`
`sesion_admin_activa()` con sesión autenticada	401 `permission denied for function`	Falta la concesión `execute` a `authenticated` en el despliegue	`13-diagnostico-sesion-admin.json`
`registrar_actividad_admin()` con sesión autenticada	200	El heartbeat real funciona	`13-diagnostico-sesion-admin.json`
`cerrar_sesion_admin()` con sesión autenticada	200	El cierre real funciona	`13-diagnostico-sesion-admin.json`

Prueba	Resultado obtenido	Interpretación	Evidencia
RPC administrativas como anónimo	401 en todas	La identidad anónima está bloqueada; el 200 del heartbeat no era anónimo	`14-rpc-anonimo.json`

- **Conclusión:** el heartbeat posterior al login estaba autenticado, porque las mismas RPC devuelven 401 sin sesión. La función `sesion_admin_activa()` tiene un problema de concesión en el despliegue. `getSession()` devuelve null en el contexto del diagnóstico, lo que podría afectar a `exigirSesionAdmin()` y a las escrituras administrativas; no se probaron escrituras por no estar autorizadas.

---

5. Tabla de resultados

N.º	Acción	Resultado esperado	Resultado obtenido	Estado	Evidencia
C1.0	Abrir `/admin` sin sesión	Acceso visible y panel oculto	Acceso visible; panel oculto	APROBADA	`01-acceso-sin-sesion.png`
C1.1	Iniciar sesión con la cuenta autorizada	Panel administrativo visible	Panel visible sin error de autenticación	APROBADA	`02-panel-autenticado.png`
C2	Comprobar que el panel se muestra correctamente	Panel y vista Mesas visibles con datos	Vista Mesas visible; 6 filas	APROBADA	`02-panel-autenticado.png`
C3	Verificar ausencia de tokens en almacenamiento	Sin claves ni cookies de autenticación	`localStorage`, `sessionStorage` y cookies vacíos	APROBADA	`03-almacenamiento.json`
C4	Dejar la sesión sin actividad ≥ 6 minutos	Transcurrir el plazo sin eventos	6 min 15 s sin interacción	APROBADA	Registro de ejecución
C5	Comprobar cierre automático	Acceso visible, panel oculto y aviso	Cierre confirmado con aviso de inactividad	APROBADA	`05-sesion-expirada.png`
C6.1	Intentar recuperar por interacción	Panel sigue oculto	Panel oculto tras el clic	APROBADA	`06-tras-interaccion.png`
C6.2	Intentar recuperar por recarga	Acceso visible, panel oculto, sin tokens, `haySesionAdmin()=false`	Todo confirmado	APROBADA	`07-tras-recarga.png`
C7	Registrar evidencias sin credenciales	Capturas y JSON sin datos sensibles	Sin correo, contraseña ni tokens	APROBADA	`docs/evidencias/`

5.1 Resumen de fase 2

Bloque	Comprobaciones	APROBADA	BLOQUEADA CORRECTAMENTE	FALLIDA
API anónima (API-01..10)	10	4	6	0
Navegación pública (NAV-01..09)	9	9	0	0
Validaciones de reserva (VAL-01..11)	11	11	0	0
Reinicio de inactividad (ACT-01..05)	5	5	0	0
Total fase 2	**35**	**29**	**6**	**0**

---

6. Evidencias

Carpeta: `docs/evidencias/`.

Archivo	Contenido
`01-acceso-sin-sesion.png`	Formulario de acceso vacío; panel oculto
`02-panel-autenticado.png`	Panel administrativo autenticado, vista Mesas, sin correo visible
`03-almacenamiento.json`	Claves de `localStorage`, `sessionStorage` y cookies; todas vacías
`05-sesion-expirada.png`	Formulario de acceso con el aviso de cierre por inactividad
`06-tras-interaccion.png`	Estado tras el clic posterior al cierre; panel oculto
`07-tras-recarga.png`	Estado tras recargar; acceso visible y panel oculto
`08-api-anonima.json`	Respuestas de las 10 consultas API con identidad anónima
`08-home-desktop.png`	Página principal en escritorio
`09-menu-movil.png`	Menú móvil abierto
`09-navegacion-publica.json`	Resultados de NAV-01..09
`10-carta.png`	Pestaña de la carta con platos
`10-validaciones-reserva.json`	Resultados de VAL-01..11
`11-modal-reserva.png`	Modal de reserva con una franja seleccionada
`11-reinicio-inactividad.json`	Resultados de ACT-01..05 y marcas de tiempo
`12-validacion-campos.png`	Errores del formulario vacío
`12-resumen-fase2-publica.json`	Resumen de API, navegación y validaciones
`13-diagnostico-sesion-admin.json`	`getSession`, `getUser`, RPC de sesión y heartbeat
`13-validacion-nombre.png`	Error de nombre demasiado corto
`14-rpc-anonimo.json`	RPC administrativas invocadas sin sesión; todas 401
`14-validacion-telefono.png`	Error de teléfono inválido
`15-validacion-capacidad.png`	Error por superar la capacidad de la mesa
`16-admin-tras-reinicio.png`	Panel visible a los 6 min del login, tras actividad válida
`17-admin-cierre-final.png`	Panel cerrado tras cinco minutos desde la última actividad
`resultados-admin-inactividad.json`	Resultados estructurados de las 9 comprobaciones de fase 1

Nota de integridad visual: `06-tras-interaccion.png` es visualmente idéntica a `05-sesion-expirada.png` porque el clic posterior al cierre no alteró el estado; se conserva como evidencia del intento. Del mismo modo, `07-tras-recarga.png` es visualmente idéntica a `01-acceso-sin-sesion.png` porque ambas muestran el formulario de acceso vacío; la diferencia se verificó en el estado lógico (`haySesionAdmin()=false`) y en el almacenamiento vacío, no en la apariencia.

Verificación de datos sensibles: se buscó el correo y la contraseña en `docs/evidencias/` y no aparecieron. Las capturas no muestran los campos completos del formulario de acceso. Las capturas `16` y `17` muestran solo el panel o el formulario, sin nombres, contactos ni reservas. Los JSON de diagnóstico registran rutas, estados HTTP y booleanos; no contienen tokens, cabeceras ni cuerpos de respuesta.

---

7. Hallazgos de inspección complementaria

Estas comprobaciones se hicieron con `curl`, sin modificar nada:

Elemento	Resultado
`/`	HTTP 200
`/admin`	HTTP 200 con `Cache-Control: no-store` y `X-Robots-Tag: noindex, nofollow`
Cabeceras de seguridad	CSP, HSTS, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy` y `Permissions-Policy` presentes

Elemento	Resultado
`/supabase/schema.sql`	HTTP 404
`/tests/e2e/konys-publico.test.js`	HTTP 404
`/docs/SEGURIDAD.md`	HTTP 404
`/js/config.js`	HTTP 200 con `no-store` (contiene la configuración pública del cliente; no se leyó su contenido)

Observaciones:

- El enlace “Admin” es visible en el menú público. No otorga permisos, pero aumenta la descubribilidad del panel.
- `admin.html` es un recurso estático descargable. La protección real está en Supabase Auth, `es_admin()`, `sesion_admin_activa()` y RLS, no en ocultar el HTML.
- El cierre se produjo con el plazo local de cinco minutos; la espera de 6 minutos y 15 segundos deja margen suficiente para observarlo.
- La actividad válida renovó el heartbeat (`registrar_actividad_admin` 200) y el panel siguió visible a los 6 minutos del login, lo que confirma que el contador se reinició. El cierre ocurrió 5 minutos después del clic, no del login.
- La API anónima solo devolvió mesas, horarios, platos y ocupación sin datos personales. `reservas`, `administradores` y `sesiones_admin_actividad` respondieron 401, igual que las RPC `es_admin`, `sesion_admin_activa`, `registrar_actividad_admin` y `cerrar_sesion_admin` sin sesión.
- **Hallazgo de despliegue (prioridad alta):** `sesion_admin_activa()` devuelve `401 permission denied for function` con un usuario autenticado, aunque `002-sesion-admin-inactividad.sql` concede `execute` a `authenticated`. El esquema local y el despliegue no coinciden. Las políticas `for all` de `mesas`, `horarios`, `platos` y `reservas` invocan `public.es_admin() and public.sesion_admin_activa()`; si la concesión falta, las escrituras administrativas y las lecturas de `reservas` (que no tienen política pública de `select`) podrían fallar con el mismo 401. No se probaron escrituras ni se abrió la vista Reservas por falta de credenciales en la sesión actual, así que el impacto debe confirmarse en un entorno aislado.
- **Riesgo a verificar:** tras el login, `getSession()` devolvió `session=false` y `getUser()` devolvió `Auth session missing!`, pese a que las RPC autenticadas respondían 200. `haySesionAdmin()` y `exigirSesionAdmin()` dependen de `getSession()`. Esto podría hacer que las operaciones administrativas rechacen la sesión con `SESION_ADMIN_EXPIRADA`. No se concluye que las escrituras fallen; se necesita una prueba autorizada en un entorno aislado. La posible causa es el uso de `persistSession:false` o una diferencia de instancia del cliente, no confirmada.

---

8. Errores encontrados

Ninguno en las comprobaciones ejecutadas. No hubo fallos de autenticación, excepciones de JavaScript visibles ni resultados contradictorios en la versión final.

- Se corrigieron dos incidencias de la propia automatización, no de la aplicación: una referencia obsoleta a las pestañas de carta (`StaleElementReferenceError`) y el criterio de ACT-04, que usaba `haySesionAdmin()` en lugar del heartbeat real.
- El `401` de `sesion_admin_activa()` es un hallazgo de despliegue, no un fallo del flujo de inactividad; el heartbeat y el cierre responden 200.
- No se intentó restablecer la contraseña ni modificar la cuenta. No se realizaron operaciones destructivas ni escrituras de datos de negocio.

---

9. Pruebas pendientes

Las siguientes acciones **no se ejecutaron** en esta fase:

- Acceso administrativo sin autenticación mediante parámetros o fragmentos de URL manipulados (C1.0 cubre el caso base sin sesión).
- Recorrido completo de todas las vistas administrativas y cierre de sesión manual.
- Verificación de la vista administrativa “Reservas” y de las operaciones de escritura (crear, editar, borrar) en un entorno aislado; es prioritaria por el 401 de `sesion_admin_activa()` y el `getSession()` nulo.

- Pruebas de RLS con un usuario autenticado **no administrador**.
- Límite de reservas por IP, XSS almacenado y otras pruebas que requieren escribir datos.

Estas pruebas no deben ejecutarse en producción si implican crear reservas, modificar mesas/platos/horarios o alterar datos reales. La comprobación de la vista “Reservas” es de solo lectura, pero no se ejecutó por falta de credenciales en la sesión actual.

---

10. Recomendaciones

1. **Corregir la concesión de** `sesion_admin_activa()` **en el despliegue.** Aplicar o verificar en Supabase:

```
revoke all on function public.sesion_admin_activa() from public, anon;
grant execute on function public.sesion_admin_activa() to authenticated;
```

Después, probar una lectura autenticada de `reservas` y una escritura controlada en un entorno aislado. Mientras la función devuelva 401, las políticas RLS que la invocan podrían bloquear operaciones administrativas.

2. **Verificar** `exigirSesionAdmin()` **con** `persistSession:false`**.** `getSession()` devolvió null tras el login aunque las RPC estaban autenticadas. Probar una escritura administrativa en un entorno aislado; si falla con `SESION_ADMIN_EXPIRADA`, ajustar el helper para usar una fuente de sesión fiable.
3. **Rotar la contraseña administrativa** utilizada en esta auditoría, porque fue compartida en el chat. Usar una contraseña única de 14+ caracteres y, si es posible, activar 2FA en Supabase y GitHub.
4. **Crear una cuenta administrativa exclusiva de pruebas** para futuras auditorías, separada de la cuenta personal.
5. **Mantener** `persistSession: false` y el heartbeat en RLS; el reinicio del contador se validó correctamente. Añadir una prueba automatizada de expiración del heartbeat del lado servidor en un entorno aislado.
6. **Conservar las cabeceras de seguridad** y la política `no-store`/`noindex` de `/admin`.
7. **Valorar quitar el enlace “Admin” del menú público** si no es necesario para los clientes; no es una vulnerabilidad, pero reduce la descubribilidad.
8. **No versionar evidencias con datos personales.** Las actuales no contienen credenciales ni datos de reservas, pero conviene revisar las capturas antes de publicarlas.
9. **Mantener las pruebas de API anónima** en CI para evitar que una política RLS futura exponga reservas o administradores.

---

11. Conclusión general

Dentro del alcance autorizado, la aplicación **cumple** con los controles probados:

- El panel administrativo no se muestra sin autenticación.
- El inicio de sesión con la cuenta autorizada funciona.
- La sesión no se persiste en `localStorage`, `sessionStorage` ni cookies.
- Tras cinco minutos sin actividad, la sesión se cierra y el panel deja de estar disponible.
- La actividad válida reinicia el contador; el panel sigue visible a los 6 minutos del login si hubo actividad a los 4 min 10 s.
- Una interacción posterior o una recarga no recuperan la sesión.
- La API anónima no expone reservas, datos personales, administradores ni sesiones.
- La navegación pública y las validaciones de reserva funcionan; los casos inválidos no invocan `crear_reserva`.
- Las evidencias generadas no exponen correo, contraseña ni tokens.

Resultado global del alcance ejecutado: 44 comprobaciones; 38 APROBADAS, 6 BLOQUEADAS CORRECTAMENTE y 0 FALLIDAS.

Queda **un hallazgo de despliegue de prioridad alta**: `sesion_admin_activa()` devuelve 401 con un usuario autenticado. El flujo de inactividad no depende de esa función para el heartbeat, pero las políticas RLS de escritura sí la invocan. Antes de considerar la aplicación completamente validada, deben corregirse la concesión y verificarse las escrituras administrativas en un entorno aislado. La auditoría completa sigue pendiente en los puntos de la sección 9.